

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Logic Design with HDL (CO1026)

Assignment Report

Designing a simple RISC Processor

Instructor: Nguyễn Thành Lộc

Students:	Trần Vinh Quang	2550342
	Lê Hiễn Vinh	2453422
	Nguyễn Đức Thiên Tân	2550345
	Đặng Ngọc Lan Anh	2550258

HO CHI MINH CITY, JUNE 2026



Contents

1	Introduction	3
1.1	Project Overview	3
1.2	Objectives	3
1.3	Tools and Environment	4
1.4	Main Features of the CPU	4
1.5	Report Organization	5
2	Theoretical Overview	5
2.1	Fetch Phase	6
2.2	Execution Phase	6
3	Design	7
3.1	Top-level System Architecture	7
3.2	Functional Blocks	8
3.2.1	Program Counter (PC)	8
3.2.2	Address Multiplexer (Address MUX)	8
3.2.3	Arithmetic Logic Unit (ALU)	8
3.2.4	Controller Unit	9
3.2.5	Instruction Register (IR)	10
3.2.6	Accumulator Register (AC)	10
3.2.7	Memory	10
4	Implementation	11
5	Testing and Evaluation	23
6	Conclusion and Future Work	40
6.1	Conclusion	40
6.2	Future Plan	41
7	Member Contribution	43

1 Introduction

1.1 Project Overview

A Central Processing Unit (CPU) is the core component of a computer system, responsible for fetching instructions, decoding them, executing operations, and controlling data movement between internal registers and memory. In this project, a simple Reduced Instruction Set Computer (RISC) CPU is designed using *Verilog Hardware Description Language (HDL)*.

The proposed CPU follows a simplified RISC architecture with a compact instruction format. Each instruction consists of a 3-bit opcode and a 5-bit operand field, allowing the processor to support eight basic instructions and address a memory space of 32 locations. Although the architecture is simple, it demonstrates the fundamental principles of processor design, including instruction fetching, instruction decoding, arithmetic and logic execution, memory access, and program control.

The CPU is organized as a set of functional hardware modules, including the Program Counter, Address Multiplexer, Memory, Instruction Register, Accumulator Register, Arithmetic Logic Unit, and Controller. These modules are designed independently and then integrated into a complete processor system. The CPU operates based on clock and reset signals and continues executing instructions until a halt instruction is encountered.

1.2 Objectives

The main objective of this project is to design, implement, and verify a simple RISC CPU at the register-transfer level using Verilog HDL. Through this project, the design process of a basic processor can be studied from both theoretical and practical perspectives.

The specific objectives of the project are as follows:

- To understand the fundamental organization and operation of a RISC-based CPU.
- To design the main functional blocks of the CPU, including the Program Counter, Address Multiplexer, Memory, Instruction Register, Accumulator Register, ALU, and Controller.
- To implement each functional block using Verilog HDL with a modular and hierarchical design approach.
- To integrate all modules into a complete CPU datapath and control path.
- To verify the correctness of each module through individual testbenches.
- To evaluate the complete CPU by observing simulation waveforms and checking the execution of supported instructions.

By completing these objectives, the project provides practical experience in digital system design, hardware modeling, finite state machine design, and simulation-based verification.

1.3 Tools and Environment

The main tool used in this project is Vivado Design Suite. Vivado provides an integrated development environment for designing, simulating, and verifying digital hardware systems described using Verilog HDL. In this project, Vivado is used to create Verilog source files, develop testbenches, run simulations, and analyze signal waveforms. The source code is formatted, linted, and maintained using **Verible** and the SystemVerilog Language Server (SVLS) to ensure strict hardware coding conventions.

Each CPU component is implemented as an independent Verilog module and verified using a corresponding testbench. A custom **Python** script is implemented with Icarus Verilog (**iverilog**) serves as the primary compiler for rapid, iterative command-line simulations and unit testing. After unit-level verification, the modules are connected together in a top-level design to perform system-level simulation. The waveform viewer in Vivado is used to observe the behavior of internal signals, such as the program counter value, memory address, instruction register output, accumulator value, ALU result, and controller signals.

Vivado is suitable for this project because it supports hierarchical hardware design, Verilog-based modeling, testbench simulation, and detailed waveform analysis. These features allow the design to be verified systematically before further hardware synthesis or implementation.

1.4 Main Features of the CPU

The designed CPU supports a small but representative set of instructions. Since the opcode field is 3 bits wide, the CPU can define up to 8 instructions. These instructions include program control, arithmetic operations, logic operations, memory access, and conditional execution.

The main features of the CPU are summarized as follows:

- A 3-bit opcode field supporting eight basic instructions.
- A 5-bit operand field supporting 32 memory address locations.
- A Program Counter used to store and update the current instruction address.
- An Address Multiplexer used to select between the instruction address and operand address.
- A Memory module used to store both instructions and data.
- An Instruction Register used to hold and decode the current instruction.

- An Accumulator Register used to store intermediate and final computation results.
- An ALU used to perform arithmetic and logic operations.
- A Controller implemented as a finite state machine to generate control signals for instruction execution.
- A halt mechanism that stops program execution when the halt instruction is detected.

The CPU performs its operation through a sequence of basic stages: instruction fetch, instruction decode, operand access, execution, and result storage. This execution flow reflects the essential behavior of a real processor while remaining simple enough for analysis, implementation, and verification.

1.5 Report Organization

This report is organized into six main sections.

Section 1 introduces the project, including the project overview, objectives, tools and environment, main CPU features, and report structure.

Section 2 presents the theoretical background of the project. It explains the basic concepts of RISC architecture, instruction format, CPU execution cycle, datapath, control path, and supported instructions.

Section 3 describes the design of the CPU. This section presents the overall system architecture, block diagram, and detailed functions of each hardware module.

Section 4 discusses the implementation of the CPU using Verilog HDL. It explains the module structure, implementation approach, controller finite state machine, memory organization, and top-level integration.

Section 5 presents the testing and evaluation process. It includes unit-level testbenches, system-level simulation, waveform analysis, and functional evaluation of the CPU.

Section 6 concludes the report by summarizing the achieved results, discussing difficulties encountered during the project, and proposing possible future improvements.

2 Theoretical Overview

Based on the project specification, the RISC CPU designed in this assignment features a 3-bit opcode and a 5-bit operand. This architecture supports 8 types of instructions (HLT, SKZ, ADD, AND, XOR, LDA, STO, JMP) and a 32-word memory address space. The processor operates synchronously with a clock signal and an active-high reset signal (the default reset state is INST_ADDR). Program execution stops upon encountering the HLT instruction.

By analyzing the controller state table and the functional requirements of each block, the CPU's operation flow can be divided into two primary phases spanning 8 clock cycles: the Fetch Phase and the Execution Phase.

Table 2.1: CPU Controller state and Control signals

Outputs	Phase								Notes
	INST_ADDR	INST_FETCH	INST_LOAD	IDLE	OP_ADDR	OP_FETCH	ALU_OP	STORE	
sel	1	1	1	1	0	0	0	0	ALU OP = 1 if opcode is ADD, AND, XOR or LDA
rd	0	1	1	1	0	ALUOP	ALUOP	ALUOP	
ld_ir	0	0	1	1	0	0	0	0	
halt	0	0	0	0	HALT	0	0	0	
inc_pc	0	0	0	0	1	0	SKZ && zero	0	
ld_ac	0	0	0	0	0	0	0	ALUOP	
ld_pc	0	0	0	0	0	0	JMP	JMP	
wr	0	0	0	0	0	0	0	STO	
data_e	0	0	0	0	0	0	STO	STO	

2.1 Fetch Phase

- **State 1: INST_ADDR.** The Program Counter (PC) provides the address of the current instruction. The Address Mux selects the PC address (**sel** = 1) to access the memory.
- **State 2: INST_FETCH.** The memory read signal (**rd** = 1) is asserted. The instruction data is retrieved from Memory.
- **State 3: INST_LOAD.** The fetched instruction is loaded into the Instruction Register (IR) via the **ld_ir** = 1 control signal.
- **State 4: IDLE.** An intermediate wait state for the system to stabilize.

2.2 Execution Phase

- **State 5: OP_ADDR.** The IR splits the instruction into a 3-bit Opcode and a 5-bit Operand Address. The Address Mux switches to select the operand address from the IR (**sel** = 0). Simultaneously, the PC increments (**inc_pc** = 1) to prepare for the next instruction cycle.

- **State 6: OP_FETCH.** If the instruction requires data from memory (e.g., ADD, AND, LDA), the CPU reads the data from the operand address in the Memory ($rd = 1$).
- **State 7: ALU_OP.** The Arithmetic Logic Unit (ALU) performs the required computation between the data fetched from Memory and the data currently held in the Accumulator (AC), based on the decoded Opcode. If the instruction is SKZ and the **zero** flag condition is met, the PC increments once more ($inc_pc = 1$) to skip the subsequent instruction.
- **State 8: STORE.** The result of the operation is either stored back into the Memory (for the STO instruction, asserting $wr = 1$ and $data_e = 1$) or loaded into the Accumulator (for arithmetic and logic instructions, asserting $ld_ac = 1$).

3 Design

3.1 Top-level System Architecture

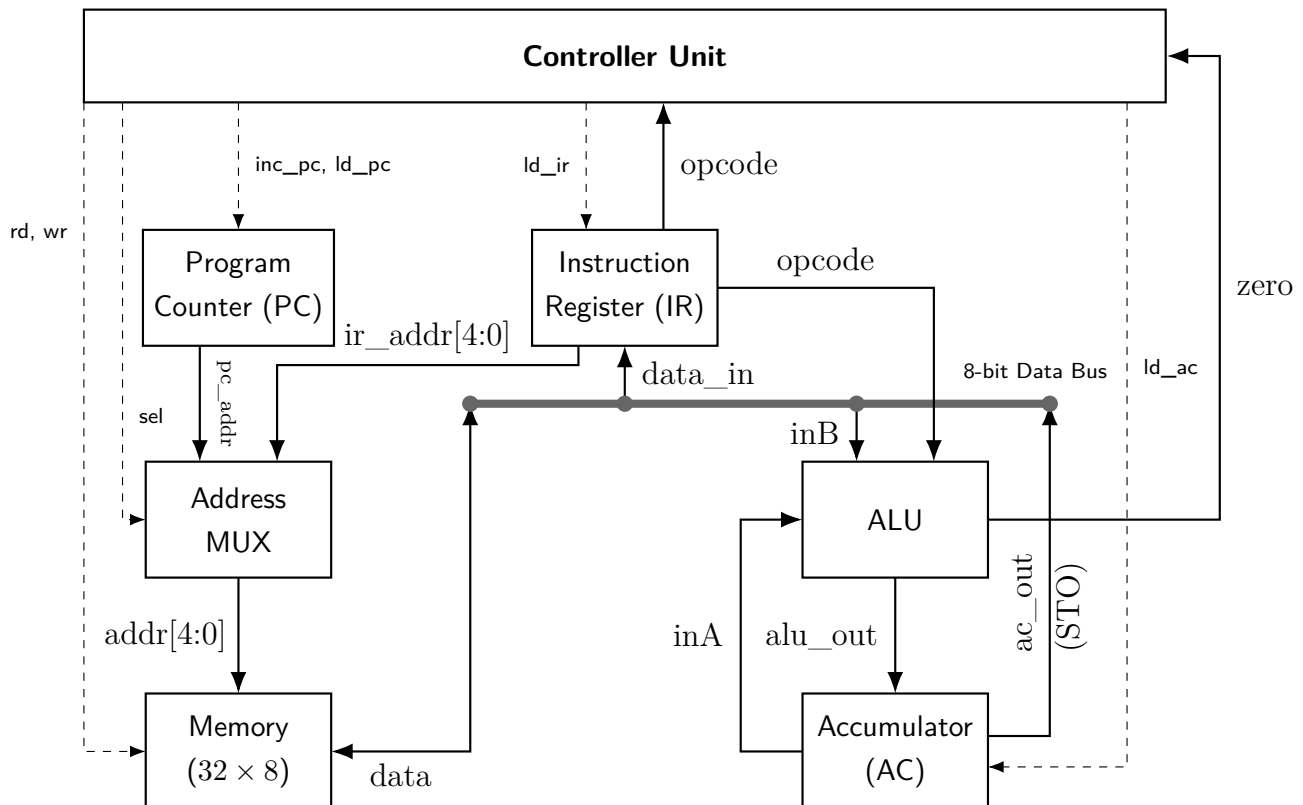


Figure 3.1: Top-Level Architecture of the RISC Processor

3.2 Functional Blocks

3.2.1 Program Counter (PC)

- **Function:**
 - The counter is a crucial component used to track and count program instructions.
 - It can also be utilized to count program states.
 - The counter operates synchronously on the rising edge of the `clk` signal.
 - The `reset` signal is active-high, clearing the counter back to 0.
 - The counter has a width of 5 bits.
 - It features a load function to force an arbitrary address into the counter. If not loading, the counter increments normally.
- **Inputs:** `clk`, `rst`, `ld_pc`, `inc_pc`, `data_in`.
- **Outputs:** `pc_out`.

3.2.2 Address Multiplexer (Address MUX)

- **Function:**
 - The Address MUX selects between the instruction address (during the instruction fetch phase) and the operand address (during the instruction execution phase).
 - The MUX has a default data width of 5 bits.
 - The width is parameterized to allow for easy modification if necessary.
- **Inputs:** `pc_addr`, `ir_addr`, `sel`.
- **Outputs:** `addr_out`.

3.2.3 Arithmetic Logic Unit (ALU)

- **Function:**
 - The ALU executes arithmetic and logical operations. The specific operation performed depends on the instruction's opcode.
 - The ALU executes 8 distinct operations on 8-bit operands (`inA` and `inB`).
 - It produces a 8-bit output and a 1-bit asynchronous `zero` flag.
 - The `zero` flag indicates whether the input `inA` is equal to 0.

- **Inputs:** inA, inB, opcode.
- **Outputs:** alu_out, zero.

Table 3.1: ALU Opcode Lookup Table

Opcode	Code	Operation Description	Output
HLT	000	Halts program execution.	inA
SKZ	001	Checks if the ALU result is equal to 0. If zero, it skips the next instruction; otherwise, execution continues normally.	inA
ADD	010	Adds the value in the Accumulator to the memory value at the instruction's address, returning the result to the Accumulator.	inA + inB
AND	011	Performs a bitwise AND on the Accumulator value and the memory value at the instruction's address.	inA AND inB
XOR	100	Performs a bitwise XOR on the Accumulator value and the memory value at the instruction's address.	inA XOR inB
LDA	101	Reads the value from the address in the instruction and loads it into the Accumulator.	inB
STO	110	Writes the Accumulator's data to the memory address specified in the instruction.	inA
JMP	111	Unconditional jump; jumps to the target address in the instruction and continues execution.	inA

3.2.4 Controller Unit

- **Function:** The central finite state machine that decodes instructions and dictates the operation of all other blocks via control signals.
- **Inputs:** clk, rst, opcode, zero.
- **Outputs:** sel, rd, ld_ir, halt, inc_pc, ld_ac, ld_pc, wr, data_e.

Table 3.2: Control Execution Scenarios by Opcode

Instruction Scenario	Active States	Expected Control Signals
HLT (000)	OP_ADDR	halt = 1
ADD, AND, XOR, LDA	OP_FETCH to STORE	rd = 1, ld_ac = 1 (at STORE phase)
STO (110)	ALU_OP, STORE	wr = 1, data_e = 1
JMP (111)	ALU_OP, STORE	ld_pc = 1

3.2.5 Instruction Register (IR)

- **Function:** As described in the reference documentation, it captures and holds the current instruction fetched from memory, splitting it into an opcode and an operand.
- **Inputs:** clk, rst, ld_ir, data_in.
- **Outputs:** opcode, operand.

3.2.6 Accumulator Register (AC)

- **Function:** As described in the reference documentation, it acts as the primary working register for ALU operations and data transfers.
- **Inputs:** clk, rst, ld_ac, data_in.
- **Outputs:** ac_out.

3.2.7 Memory

- **Function:**
 - The memory module stores both instructions and data.
 - It is designed to separate read and write functions while utilizing a single bidirectional data port. Reading and writing cannot occur simultaneously.
 - Configured for 5-bit addresses and 32-bit data.
 - Uses 1-bit control signals to enable reading (**rd**) and writing (**wr**).
 - Memory operations are synchronized to the rising edge of the clk signal.
- **Inputs:** clk, rd, wr, addr.
- **Inout:** data.

4 Implementation

```
1 module PC (  
2     input          clk,  
3     input          rst,  
4     input          ld_pc,  
5     input          inc_pc,  
6     input [4:0] data_in,  
7     output reg [4:0] pc_out  
8 );  
9  
10 always @(posedge clk) begin  
11     if (rst) begin  
12         pc_out <= 5'd0;  
13     end else if (ld_pc) begin  
14         pc_out <= data_in;  
15     end else if (inc_pc) begin  
16         pc_out <= pc_out + 5'd1;  
17     end else begin  
18         pc_out <= pc_out;  
19     end  
20 end  
21  
22 endmodule
```

Listing 4.1: Program Counter Module

```
1 module address_mux #(  
2     parameter WIDTH = 5  
3 ) (  
4     input [WIDTH-1:0] pc_addr,  
5     input [WIDTH-1:0] ir_addr,  
6     input          sel,  
7     output [WIDTH-1:0] addr_out  
8 );  
9     assign addr_out = (sel) ? pc_addr : ir_addr;  
10 endmodule
```

Listing 4.2: Address MUX Module

```
1 module ALU (  
2     input [2:0] opcode,  
3     input [7:0] inA,  
4     input [7:0] inB,  
5     output reg [7:0] alu_out,  
6     output zero  
7 );  
8 always @(*) begin  
9     case (opcode)  
10        3'b000: alu_out = inA;           //HLT  
11        3'b001: alu_out = inA;           //SKZ  
12        3'b010: alu_out = inA + inB;     //ADD  
13        3'b011: alu_out = inA & inB;     //AND  
14        3'b100: alu_out = inA ^ inB;     //XOR  
15        3'b101: alu_out = inB;           //LDA  
16        3'b110: alu_out = inA;           //STO  
17        3'b111: alu_out = inA;           //JMP  
18        default: alu_out = 8'b00000000;  
19    endcase  
20 end  
21 assign zero = (inA == 8'b00000000);  
22 endmodule
```

Listing 4.3: ALU Module



```
1 module controller (  
2     input clk,  
3     input rst,  
4     input [2:0] opcode,  
5     input zero,  
6  
7     output reg sel,  
8     output reg rd,  
9     output reg ld_ir,  
10    output reg halt,  
11    output reg inc_pc,  
12    output reg ld_ac,  
13    output reg ld_pc,  
14    output reg wr,  
15    output reg data_e  
16  
17 );  
18     localparam INST_ADDR=3'd0,  
19                INST_FETCH=3'd1,  
20                INST_LOAD=3'd2,  
21                IDLE=3'd3,  
22                OP_ADDR=3'd4,  
23                OP_FETCH=3'd5,  
24                ALU_OP=3'd6,  
25                STORE=3'd7;  
26  
27     // Opcodes  
28     localparam HLT = 3'b000,  
29                SKZ = 3'b001,  
30                ADD = 3'b010,  
31                AND = 3'b011,  
32                XOR = 3'b100,  
33                LDA = 3'b101,  
34                STO = 3'b110,  
35                JMP = 3'b111;  
36  
37     reg [2:0] state;  
38     reg halted;
```

```
39
40 wire aluop;
41 assign aluop = (opcode == ADD) ||
42               (opcode == AND) ||
43               (opcode == XOR) ||
44               (opcode == LDA);
45
46 // State register
47 always @(posedge clk) begin
48     if (rst) begin
49         state <= INST_ADDR;
50         halted <= 1'b0;
51     end else if (halted) begin
52         state <= state;
53         halted <= 1'b1;
54     end else begin
55         if ((state == OP_ADDR) && (opcode == HLT)) begin
56             state <= OP_ADDR;
57             halted <= 1'b1;
58         end else begin
59             state <= (state == STORE) ? INST_ADDR : state + 3'd1;
60         end
61     end
62 end
63
64 // Output logic (combinational)
65 always @(*) begin
66     // Default values
67     sel      = 1'b0;
68     rd       = 1'b0;
69     ld_ir    = 1'b0;
70     halt     = 1'b0;
71     inc_pc   = 1'b0;
72     ld_ac    = 1'b0;
73     ld_pc    = 1'b0;
74     wr       = 1'b0;
75     data_e   = 1'b0;
76
77     if (rst) begin
```



```
78     sel = 1'b1;
79 end else if (halted) begin
80     halt = 1'b1;
81 end else begin
82     case (state)
83
84         INST_ADDR: begin
85             sel = 1'b1;
86         end
87
88         INST_FETCH: begin
89             sel = 1'b1;
90             rd  = 1'b1;
91         end
92
93         INST_LOAD: begin
94             sel  = 1'b1;
95             rd   = 1'b1;
96             ld_ir = 1'b1;
97         end
98
99         IDLE: begin
100             sel  = 1'b1;
101             rd   = 1'b1;
102             ld_ir = 1'b1;
103         end
104
105         OP_ADDR: begin
106             inc_pc = 1'b1;
107
108             if (opcode == HLT) begin
109                 halt = 1'b1;
110             end
111         end
112
113         OP_FETCH: begin
114             rd = aluop;
115         end
116
```

```
117     ALU_OP: begin
118         rd = aluop;
119
120         if (opcode == SKZ && zero) begin
121             inc_pc = 1'b1;
122         end
123
124         if (opcode == JMP) begin
125             ld_pc = 1'b1;
126         end
127
128         if (opcode == ST0) begin
129             data_e = 1'b1;
130         end
131     end
132
133     STORE: begin
134         rd      = aluop;
135         ld_ac   = aluop;
136
137         if (opcode == JMP) begin
138             ld_pc = 1'b1;
139         end
140
141         if (opcode == ST0) begin
142             wr      = 1'b1;
143             data_e = 1'b1;
144         end
145     end
146
147     default: begin
148         sel      = 1'b0;
149         rd       = 1'b0;
150         ld_ir    = 1'b0;
151         halt     = 1'b0;
152         inc_pc   = 1'b0;
153         ld_ac    = 1'b0;
154         ld_pc    = 1'b0;
155         wr       = 1'b0;
```




```
156         data_e = 1'b0;  
157     end  
158  
159     endcase  
160 end  
161 end  
162  
163 endmodule
```

Listing 4.4: Controller Unit Module

```
1 module instruction_register (  
2     input      clk,  
3     input      rst,  
4     input      ld_ir,  
5     input  [7:0] data_in,  
6     output [ 2:0] opcode,  
7     output [ 4:0] operand  
8 );  
9     reg [7:0] ir_reg;  
10    always @(posedge clk) begin  
11        if (rst) begin  
12            ir_reg <= 8'b00000000;  
13        end  
14        else if (ld_ir) begin  
15            ir_reg <= data_in[7:0];  
16        end  
17    end  
18    assign opcode = ir_reg[7:5];  
19    assign operand = ir_reg[4:0];  
20 endmodule
```

Listing 4.5: Controller Unit Module

```
1 module accumulator (  
2     input      clk,  
3     input      rst,  
4     input  [7:0] data_in,  
5     input      ld_ac,  
6     output reg [7:0] ac_out  
7 );  
8     always @(posedge clk) begin  
9         if (rst) ac_out <= 8'b00000000;  
10        else if (ld_ac) ac_out <= data_in;  
11        else ac_out <= ac_out;  
12    end  
13 endmodule
```

Listing 4.6: Accumulator Register Module

```
1 module Memory (  
2     input      clk,  
3     input      rd,  
4     input      wr,  
5     input [4:0] addr,  
6     inout [7:0] data  
7 );  
8  
9     reg [7:0] mem_cells[31:0];  
10    reg [7:0] data_out;  
11  
12    always @(posedge clk) begin  
13        if (wr && !rd) begin  
14            mem_cells[addr] <= data;  
15        end else  
16            if (rd && !wr) begin  
17                data_out <= mem_cells[addr];  
18            end  
19        end  
20  
21        assign data = (rd && !wr) ? data_out : 8'hZZ;  
22  
23    endmodule
```

Listing 4.7: Memory Module



```
1 module CPU (  
2     input  clk,  
3     input  rst,  
4     output halt  
5 );  
6  
7     // Control signals  
8     wire sel;  
9     wire rd;  
10    wire ld_ir;  
11    wire inc_pc;  
12    wire ld_ac;  
13    wire ld_pc;  
14    wire wr;  
15    wire data_e;  
16  
17    // Datapath signals  
18    wire [4:0] pc_out;  
19    wire [4:0] operand;  
20    wire [4:0] mem_addr;  
21  
22    wire [2:0] opcode;  
23  
24    wire [7:0] data_bus;  
25    wire [7:0] ac_out;  
26    wire [7:0] alu_out;  
27  
28    wire zero;  
29  
30    // Program Counter  
31    PC u_pc (  
32        .clk      (clk),  
33        .rst      (rst),  
34        .ld_pc    (ld_pc),  
35        .inc_pc   (inc_pc),  
36        .data_in  (operand),  
37        .pc_out   (pc_out)  
38    );
```



```
39
40 // Address MUX
41 address_mux #(
42     .WIDTH(5)
43 ) u_address_mux (
44     .pc_addr (pc_out),
45     .ir_addr (operand),
46     .sel      (sel),
47     .addr_out(mem_addr)
48 );
49
50 // Memory
51 Memory u_memory (
52     .clk (clk),
53     .rd  (rd),
54     .wr  (wr),
55     .addr(mem_addr),
56     .data(data_bus)
57 );
58
59 // Instruction Register
60 instruction_register u_ir (
61     .clk      (clk),
62     .rst      (rst),
63     .ld_ir    (ld_ir),
64     .data_in  (data_bus),
65     .opcode   (opcode),
66     .operand  (operand)
67 );
68
69 // Accumulator
70 accumulator u_ac (
71     .clk      (clk),
72     .rst      (rst),
73     .data_in  (alu_out),
74     .ld_ac    (ld_ac),
75     .ac_out   (ac_out)
76 );
77
```

```
78  // ALU
79  ALU u_alu (
80      .opcode (opcode),
81      .inA     (ac_out),
82      .inB     (data_bus),
83      .alu_out (alu_out),
84      .zero    (zero)
85  );
86
87  // Controller
88  controller u_controller (
89      .clk      (clk),
90      .rst      (rst),
91      .opcode   (opcode),
92      .zero     (zero),
93
94      .sel      (sel),
95      .rd       (rd),
96      .ld_ir    (ld_ir),
97      .halt     (halt),
98      .inc_pc   (inc_pc),
99      .ld_ac    (ld_ac),
100     .ld_pc    (ld_pc),
101     .wr       (wr),
102     .data_e   (data_e)
103 );
104
105 endmodule
```

Listing 4.8: CPU Module

5 Testing and Evaluation

```
1 `timescale 1ns / 1ps
2 module PC_tb;
3
4     reg        clk;
5     reg        rst;
6     reg        ld_pc;
7     reg        inc_pc;
8     reg [4:0]  data_in;
9     wire [4:0] pc_out;
10
11     PC uut (
12         .clk      (clk),
13         .rst      (rst),
14         .ld_pc    (ld_pc),
15         .inc_pc   (inc_pc),
16         .data_in  (data_in),
17         .pc_out   (pc_out)
18     );
19
20     initial clk = 0;
21     always #5 clk = ~clk;
22
23     task tick;
24         input [63:0] cycle;
25         begin
26             @(posedge clk);
27             #1;
28             $display("cycle=%0d | rst=%b | ld_pc=%b | inc_pc=%b | data_in=%0d
29                     | pc_out=%0d", cycle, rst,
30                         ld_pc, inc_pc, data_in, pc_out);
31         end
32     endtask
33
34     initial begin
35         rst = 1;
36         ld_pc = 0;
37         inc_pc = 0;
```

```
37     data_in = 5'd0;
38
39     $display("--- TC1: Reset ---");
40     tick(1); // pc_out = 0
41     rst = 0;
42
43     $display("--- TC2: Increment ---");
44     inc_pc = 1;
45     tick(2); // pc_out = 1
46     tick(3); // pc_out = 2
47     tick(4); // pc_out = 3
48     inc_pc = 0;
49
50     $display("--- TC3: Load ---");
51     data_in = 5'd20;
52     ld_pc = 1;
53     tick(5); // pc_out = 20
54     ld_pc = 0;
55     tick(6); // hold 20
56
57     $display("--- TC4: Priority rst > ld_pc > inc_pc ---");
58     data_in = 5'd25;
59     rst = 1;
60     ld_pc = 1;
61     inc_pc = 1;
62     tick(7); // reset dominates -> 0
63     rst = 0;
64     tick(8); // ld_pc dominates -> 25
65     ld_pc = 0;
66     tick(9); // inc_pc -> 26
67     inc_pc = 0;
68
69     $display("--- TC5: Hold State ---");
70     tick(10); // hold 26
71     tick(11); // hold 26
72
73     $display("--- TC6: 5-bit Wrap-around ---");
74     data_in = 5'd31;
75     ld_pc = 1;
```



```
76     tick(12);    // pc_out = 31
77     ld_pc  = 0;
78     inc_pc = 1;
79     tick(13);    // pc_out wraps to 0
80     inc_pc = 0;
81
82     $display("--- TC7: Jump & Increment ---");
83     data_in = 5'd10;
84     ld_pc   = 1;
85     tick(14);    // pc_out = 10
86     ld_pc   = 0;
87     inc_pc = 1;
88     tick(15);    // 11
89     tick(16);    // 12
90     inc_pc = 0;
91
92     $display("--- TC8: Reset from Non-Zero Value ---");
93     data_in = 5'd30;
94     ld_pc   = 1;
95     tick(17);    // pc_out = 30
96     ld_pc = 0;
97     rst     = 1;
98     tick(18);    // pc_out = 0
99     rst     = 0;
100
101     $display("--- TC9: Data Noise Immunity ---");
102     inc_pc = 1;
103     tick(19);    // pc_out = 1
104     data_in = 5'd31;
105     tick(20);    // pc_out = 2, not affected by data_in
106     inc_pc = 0;
107
108     $display("--- DONE ---");
109     $finish;
110 end
111
112 endmodule
```

Listing 5.1: Program Counter testbench

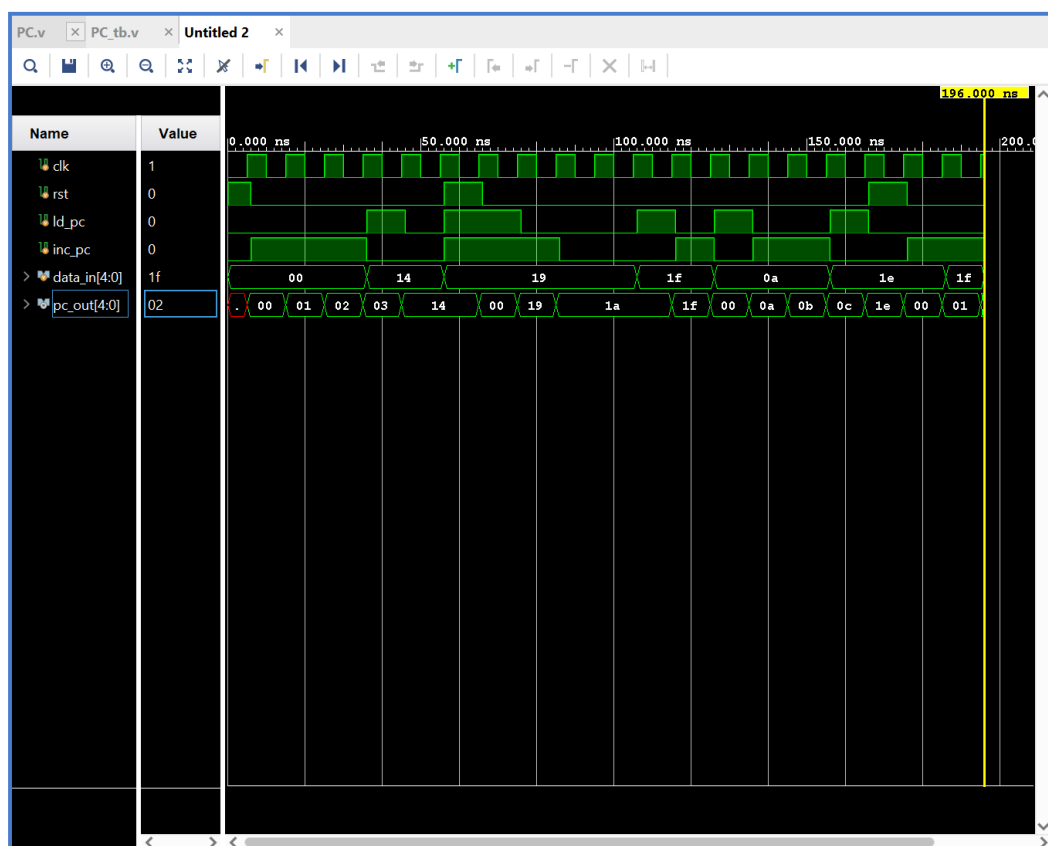


Figure 5.1: Waveform of PC testbench

```
1 `timescale 1ns / 1ps
2 module ALU_tb;
3
4     reg    [7:0] inA;
5     reg    [7:0] inB;
6     reg    [2:0] opcode;
7
8     wire   [7:0] alu_out;
9     wire           zero;
10
11     // DUT (Device Under Test)
12     ALU uut (
13         .inA      (inA),
14         .inB      (inB),
15         .opcode   (opcode),
16         .alu_out  (alu_out),
17         .zero     (zero)
18     );
19
20     task display_state;
21         input [8*6:1] tc_name;
22         begin
23             #1;
24             $display("%s | opcode=%b | inA=%0d | inB=%0d | zero=%b | alu_out
25                     =%0d",
26                     tc_name, opcode, inA, inB, zero, alu_out);
27         end
28     endtask
29
30     initial begin
31         // Initialize
32         inA = 0;
33         inB = 0;
34         opcode = 3'b000;
35         #10;
36
37         // TC1: Zero Status Flag
38         $display("\n--- TC1: Zero Status Flag ---");
```

```
38     inA = 8'd0;          display_state("TC1.1"); // zero = 1
39     inA = 8'd50;        display_state("TC1.2"); // zero = 0
40
41     // TC2: Arithmetic & Logic Operations
42     $display("\n--- TC2: Arithmetic & Logic ---");
43     inA = 8'd100; inB = 8'd25;
44     opcode = 3'b010; display_state("ADD"); // 125
45     opcode = 3'b011; display_state("AND"); // 0
46     opcode = 3'b100; display_state("XOR"); // 125
47     opcode = 3'b101; display_state("LDA"); // 25
48
49     // TC3: Transfer Instructions
50     $display("\n--- TC3: HLT / SKZ / STO / JMP ---");
51     inA = 8'd200; inB = 8'd100;
52     opcode = 3'b000; display_state("HLT"); // 200
53     opcode = 3'b001; display_state("SKZ"); // 200
54     opcode = 3'b110; display_state("STO"); // 200
55     opcode = 3'b111; display_state("JMP"); // 200
56
57     // TC4: Combinational Property
58     $display("\n--- TC4: Combinational Property ---");
59     opcode = 3'b010; // ADD
60     inA = 8'd10; inB = 8'd20; #1;
61     $display("TC4.1 | alu_out=%0d (Expected 30)", alu_out);
62     inA = 8'd50; inB = 8'd10; #1;
63     $display("TC4.2 | alu_out=%0d (Expected 60)", alu_out);
64
65     // TC5: Boundary Cases
66     $display("\n--- TC5: Boundary Cases ---");
67     opcode = 3'b010; // ADD
68     inA = 8'hFF; inB = 8'h01; display_state("OVF1"); // Expect 0
69     inA = 8'h7F; inB = 8'h7F; display_state("OVF2"); // Expect 254
70
71     // TC6: Complex Bitwise Logic
72     $display("\n--- TC6: Complex Bitwise Logic ---");
73     inA = 8'hAA; inB = 8'h55; // 10101010 vs 01010101
74     opcode = 3'b011; display_state("AND_C"); // 0
75     opcode = 3'b100; display_state("XOR_C"); // 255
76
```



```
77 // TC7: Zero Flag Independence
78 $display("\n--- TC7: Zero Flag Independence ---");
79 opcode = 3'b101; // LDA
80 inA = 8'd0; inB = 8'd100; #1;
81 $display("TC7.1 | zero=%b | alu_out=%0d", zero, alu_out);
82 inA = 8'd1; inB = 8'd100; #1;
83 $display("TC7.2 | zero=%b | alu_out=%0d", zero, alu_out);
84
85 $display("\n--- ALL TESTS FINISHED ---");
86 $finish;
87 end
88
89 endmodule
```

Listing 5.2: ALU testbench

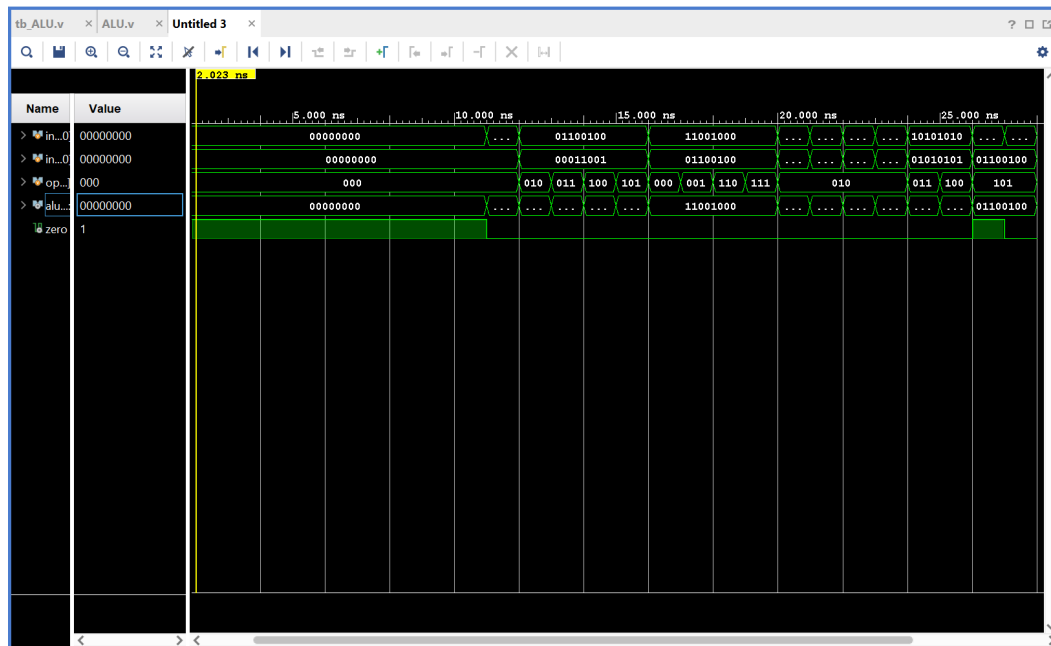


Figure 5.2: Waveform of ALU testbench

```

1  `timescale 1ns / 1ps
2  module Controller_tb;
3
4      reg      clk;
5      reg      rst;
6      reg [2:0] opcode;
7      reg      zero;
8
9      wire sel, rd, ld_ir, halt, inc_pc, ld_ac, ld_pc, wr, data_e;
10
11     controller uut (
12         .clk      (clk),
13         .rst      (rst),
14         .opcode(opcode),
15         .zero     (zero),
16         .sel      (sel),
17         .rd       (rd),
18         .ld_ir    (ld_ir),
19         .halt     (halt),
20         .inc_pc   (inc_pc),
21         .ld_ac    (ld_ac),
22         .ld_pc    (ld_pc),

```

```
23     .wr      (wr),
24     .data_e(data_e)
25 );
26
27 initial clk = 0;
28 always #5 clk = ~clk;
29
30 task tick;
31     begin
32         @(posedge clk);
33         #1;
34     end
35 endtask
36
37 task reset_controller;
38     begin
39         rst = 1;
40         repeat (2) @(posedge clk);
41         #1;
42         rst = 0;
43     end
44 endtask
45
46 task display_state;
47     input [8*14:1] state_name;
48     begin
49         $display(
50             "%s | state=%3b opcode=%3b zero=%b | sel=%b rd=%b ld_ir=%b
51             halt=%b inc_pc=%b ld_ac=%b ld_pc=%b wr=%b data_e=%b",
52             state_name, uut.state, opcode, zero, sel, rd, ld_ir, halt,
53             inc_pc, ld_ac, ld_pc, wr,
54             data_e);
55     end
56 endtask
57
58 initial begin
59     opcode = 3'b000;
60     zero   = 1'b0;
61     rst    = 1'b1;
```

```
60
61     reset_controller();
62
63     $display("--- TC1: System Reset / INST_ADDR ---");
64     display_state("0:INST_ADDR    ");
65
66     $display("--- TC2: Fetch Phase ---");
67     opcode = 3'b010;    // ADD
68     display_state("0:INST_ADDR    ");
69     tick();
70     display_state("1:INST_FETCH   ");
71     tick();
72     display_state("2:INST_LOAD    ");
73     tick();
74     display_state("3:IDLE         ");
75
76     $display("--- TC3: Execution Phase (ADD) ---");
77     tick();
78     display_state("4:OP_ADDR      ");
79     tick();
80     display_state("5:OP_FETCH     ");
81     tick();
82     display_state("6:ALU_OP       ");
83     tick();
84     display_state("7:STORE        ");
85     tick();
86     display_state("0:INST_ADDR    ");
87
88     $display("--- TC4: Execution Phase (STO) ---");
89     reset_controller();
90     opcode = 3'b110;    // STO
91     tick();
92     tick();
93     tick();
94     tick();
95     display_state("4:OP_ADDR      ");
96     tick();
97     display_state("5:OP_FETCH     ");
98     tick();
```




```
99     display_state("6:ALU_OP      ");
100     tick();
101     display_state("7:STORE      ");
102
103     $display("--- TC5: Execution Phase (JMP) ---");
104     reset_controller();
105     opcode = 3'b111; // JMP
106     tick();
107     tick();
108     tick();
109     tick();
110     display_state("4:OP_ADDR      ");
111     tick();
112     display_state("5:OP_FETCH      ");
113     tick();
114     display_state("6:ALU_OP      ");
115     tick();
116     display_state("7:STORE      ");
117
118     $display("--- TC6: Execution Phase (HLT, stable halt) ---");
119     reset_controller();
120     opcode = 3'b000; // HLT
121     tick();
122     tick();
123     tick();
124     tick();
125     display_state("4:OP_ADDR/HLT ");
126     tick();
127     display_state("HOLD_HALT_1    ");
128     tick();
129     display_state("HOLD_HALT_2    ");
130
131     $display("--- TC7.1: SKZ zero=0, no skip ---");
132     reset_controller();
133     opcode = 3'b001; // SKZ
134     zero    = 1'b0;
135     tick();
136     tick();
137     tick();
```

```
138     tick();
139     display_state("4:OP_ADDR      ");
140     tick();
141     display_state("5:OP_FETCH    ");
142     tick();
143     display_state("6:ALU_OP      "); // inc_pc should be 0
144     tick();
145     display_state("7:STORE       ");
146
147     $display("--- TC7.2: SKZ zero=1, skip next instruction ---");
148     reset_controller();
149     opcode = 3'b001; // SKZ
150     zero    = 1'b1;
151     tick();
152     tick();
153     tick();
154     tick();
155     display_state("4:OP_ADDR      ");
156     tick();
157     display_state("5:OP_FETCH    ");
158     tick();
159     display_state("6:ALU_OP      "); // inc_pc should be 1
160     tick();
161     display_state("7:STORE       ");
162
163     $display("--- DONE ---");
164     $finish;
165 end
166
167 endmodule
```

Listing 5.3: Controller testbench

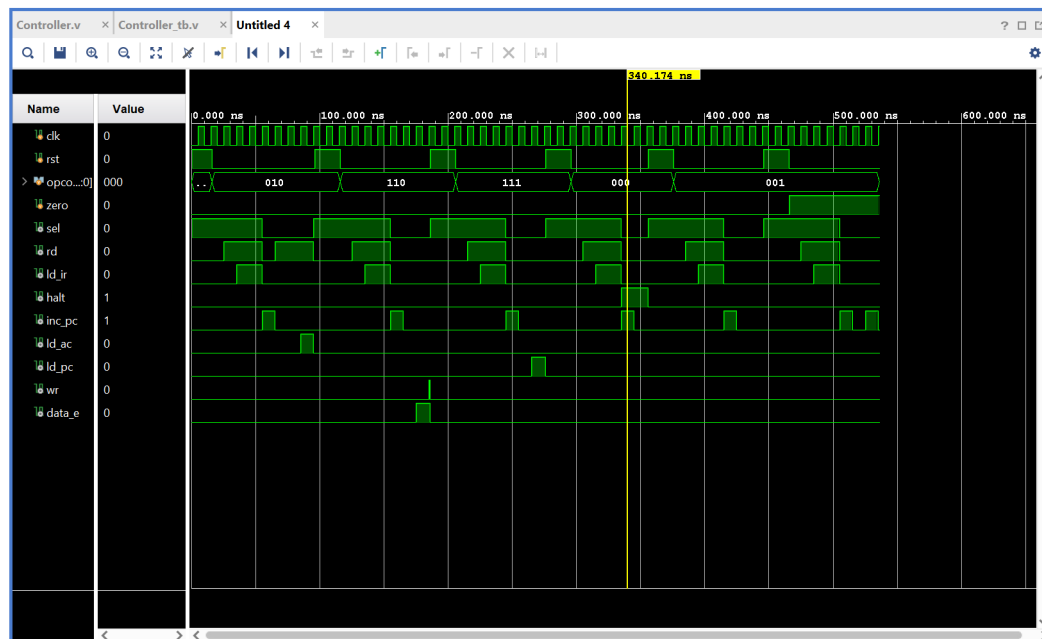


Figure 5.3: Waveform of Controller testbench

```

1  `timescale 1ns / 1ps
2  module CPU_tb;
3      // Signals
4      reg clk;
5      reg rst;
6      wire halt;
7
8      // DUT instantiation
9      CPU uut (
10         .clk (clk),
11         .rst (rst),
12         .halt(halt)
13     );
14
15     // Clock: 10 ns period
16     initial clk = 0;
17     always #5 clk = ~clk;
18
19     // Task: display CPU state
20     task display_cpu_state;
21     begin
22         $display("time=%t | PC=%2d | State=%3b | Op=%3b | AC=%0d | Halt=%

```

```
23         b", $time, uut.u_pc.pc_out,
24             uut.u_controller.state, uut.opcode, uut.u_ac.ac_out,
25             halt);
26
27     end
28 endtask
29
30 // Load Program & Control Simulation
31 initial begin
32     // 1. Load specific test program from README
33     // LDA 20, ADD 21, STO 22, HLT
34     uut.u_memory.mem_cells[0] = 8'hB4; // LDA 20 (Opcode 101, Operand
35         10100)
36     uut.u_memory.mem_cells[1] = 8'h55; // ADD 21 (Opcode 010, Operand
37         10101)
38     uut.u_memory.mem_cells[2] = 8'hD6; // STO 22 (Opcode 110, Operand
39         10110)
40     uut.u_memory.mem_cells[3] = 8'h00; // HLT (Opcode 000, Operand
41         00000)
42
43     // 2. Load input data according to README
44     uut.u_memory.mem_cells[20] = 8'd5;
45     uut.u_memory.mem_cells[21] = 8'd3;
46     uut.u_memory.mem_cells[22] = 8'd0; // Location to store result
47
48     // 3. Reset system
49     rst = 1;
50     repeat (2) @(posedge clk);
51     #1;
52     rst = 0;
53
54     $display("--- CPU INTEGRATION TEST STARTED ---");
55 end
56
57 // Monitor state at each positive clock edge
58 always @(posedge clk) begin
59     if (!rst) display_cpu_state();
60 end
61
62 // Final result check after HLT
```

```
56 initial begin
57     wait (halt === 1'b1);
58     repeat (2) @(posedge clk);
59
60     $display("--- FINAL RESULT CHECK ---");
61     $display("Mem[22] (Expected 8): %0d", uut.u_memory.mem_cells[22]);
62
63     if (uut.u_memory.mem_cells[22] == 8) $display("--- TEST STATUS:
64         PASS ---");
65     else $display("--- TEST STATUS: FAIL ---");
66
67     $finish;
68 end
69
70 // Safety Timeout
71 initial #10000 $finish;
72 endmodule
```

Listing 5.4: CPU testbench

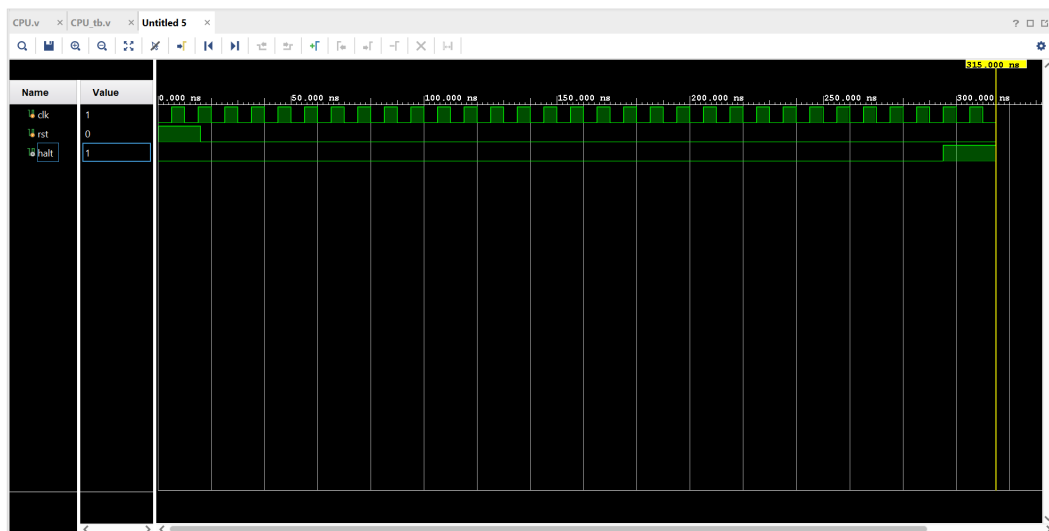


Figure 5.4: Waveform of CPU testbench



```
1 `timescale 1ns / 1ps
2 module CPU2_tb;
3
4     // Signals
5     reg clk;
6     reg rst;
7     wire halt;
8
9     // DUT instantiation
10    CPU uut (
11        .clk (clk),
12        .rst (rst),
13        .halt(halt)
14    );
15
16    // Clock: 10 ns period
17    initial clk = 0;
18    always #5 clk = ~clk;
19
20    // Task: display CPU state
21    // Prints out whenever PC changes or an instruction cycle finishes (8
22    // phases)
23    task display_cpu_state;
24    begin
25        $display("time=%t | PC=%2d | State=%3b | Op=%3b | AC=%0d | Halt=%
26        b", $time, uut.u_pc.pc_out,
27        uut.u_controller.state, uut.opcode, uut.u_ac.ac_out,
28        halt);
29    end
30    endtask
31
32    // Load Program and Control Simulation
33    initial begin
34        // 1. Load comprehensive test program into Memory
35        // Program executes: (5 + 3), XOR 5, AND 3, tests SKZ/JMP
36        uut.u_memory.mem_cells[0] = 8'hB4; // LDA 20 -> AC = 5
37        uut.u_memory.mem_cells[1] = 8'h55; // ADD 21 -> AC = 8
38        uut.u_memory.mem_cells[2] = 8'hD6; // STO 22 -> Mem[22] = 8
```

```
36    uut.u_memory.mem_cells[3] = 8'h20; // SKZ    -> AC=8 (!=0) -> No
      jump
37    uut.u_memory.mem_cells[4] = 8'hE6; // JMP 6    -> PC jumps to 6
38    uut.u_memory.mem_cells[5] = 8'h55; // ADD 21 -> (Skipped due to
      JMP)
39    uut.u_memory.mem_cells[6] = 8'h94; // XOR 20 -> 8 XOR 5 = 13
40    uut.u_memory.mem_cells[7] = 8'h75; // AND 21 -> 13 AND 3 = 1
41    uut.u_memory.mem_cells[8] = 8'hD7; // STO 23 -> Mem[23] = 1
42    uut.u_memory.mem_cells[9] = 8'hB8; // LDA 24 -> AC = 0 (Data at 24
      is 0)
43    uut.u_memory.mem_cells[10] = 8'h20; // SKZ    -> AC=0 -> Skip next
      instruction
44    uut.u_memory.mem_cells[11] = 8'hD9; // STO 25 -> (Skipped due to
      SKZ)
45    uut.u_memory.mem_cells[12] = 8'h00; // HLT    -> Halt
46
47    // 2. Load operand data
48    uut.u_memory.mem_cells[20] = 8'd5;
49    uut.u_memory.mem_cells[21] = 8'd3;
50    uut.u_memory.mem_cells[22] = 8'd0;
51    uut.u_memory.mem_cells[23] = 8'd0;
52    uut.u_memory.mem_cells[24] = 8'd0;
53
54    // 3. Reset system
55    rst = 1;
56    repeat (2) @(posedge clk);
57    #1;
58    rst = 0;
59
60    $display("--- CPU INTEGRATION TEST STARTED ---");
61    end
62
63    // Monitor state at each positive clock edge
64    always @(posedge clk) begin
65        if (!rst) display_cpu_state();
66    end
67
68    // Final result check after HLT
69    initial begin
```

```
70     wait (halt === 1'b1);
71     repeat (2) @(posedge clk);
72
73     $display("--- FINAL RESULT CHECK ---");
74     $display("Mem[22] (Expected 8): %0d", uut.u_memory.mem_cells[22]);
75     $display("Mem[23] (Expected 1): %0d", uut.u_memory.mem_cells[23]);
76
77     if (uut.u_memory.mem_cells[22] == 8 && uut.u_memory.mem_cells[23]
78         == 1)
79         $display("--- TEST STATUS: PASS ---");
80     else $display("--- TEST STATUS: FAIL ---");
81
82     $finish;
83 end
84
85 // Safety Timeout
86 initial #10000 $finish;
87
88 endmodule
```

Listing 5.5: CPU2 testbench

6 Conclusion and Future Work

6.1 Conclusion

In this assignment, our group designed and implemented a simple RISC processor using Verilog HDL. The processor follows an accumulator-based architecture and uses an 8-bit instruction format, consisting of a 3-bit opcode and a 5-bit operand field. Based on this format, the CPU supports eight instructions: HLT, SKZ, ADD, AND, XOR, LDA, STO, and JMP. The design also includes a 5-bit program counter and a 32-location memory space, which are consistent with the project specification.

The processor was developed using a modular design approach. Core components, including the Program Counter, Address Multiplexer, Memory, Instruction Register, Accumulator Register, ALU, and Controller, were implemented as independent Verilog modules and then integrated into the top-level CPU module. This organization made the datapath and control path clearer, while also making simulation, debugging, and system integration more manageable.

Key part of the design was the Controller, which was implemented as a finite state machine with

eight states: INST_ADDR, INST_FETCH, INST_LOAD, IDLE, OP_ADDR, OP_FETCH, ALU_OP, and STORE. These states coordinate the main phases of instruction execution, including instruction fetch, decoding, operand access, ALU operation, accumulator update, memory write-back, program counter update, and halt control. Through simulation, the processor was verified to execute the supported instructions correctly at both module and system levels.

The main strength of the design is its clear separation between datapath components and control logic. Since each module was tested independently before being connected in the complete CPU, functional errors were easier to identify and correct. In addition, the use of both module-level and CPU-level testbenches helped confirm the correctness of individual blocks as well as the overall instruction execution flow.

However, the current processor still has several limitations. The instruction set is relatively small and does not support more complex arithmetic or comparison operations. The 5-bit address bus also limits the memory space to only 32 locations, which restricts the size of executable programs. Moreover, the controller follows a fixed multi-state execution sequence, meaning that some simple instructions may require more clock cycles than necessary. The project also focuses mainly on functional simulation, while synthesis results, timing delay, resource utilization, and power consumption have not yet been analyzed in detail.

6.2 Future Plan

Based on these limitations, several improvements can be considered for future development. First, the instruction set can be extended to improve the flexibility of the processor. Additional instructions such as SUB, MUL, DIV, shift operations, and comparison instructions would allow the CPU to support more diverse computational tasks. For example, comparison and shift instructions would make the processor more capable of handling conditional operations and bit-level data manipulation.

Second, the memory system can be expanded by increasing the address width. Since the current 5-bit address bus only supports 32 memory locations, a wider address bus would allow the processor to access a larger program and data memory space. This improvement would make it possible to execute longer instruction sequences and store more intermediate data during program execution.

Third, the Controller can be optimized to reduce unnecessary execution cycles. In the current design, instructions follow a fixed sequence of states, even when some operations do not require all stages. Future versions could use instruction-dependent state transitions so that simpler instructions, such as HLT or JMP, complete in fewer cycles. More advanced control techniques, including pipelining and hazard handling, could also be explored to improve instruction throughput.

Fourth, verification should be extended with more comprehensive test programs. Although



module-level and CPU-level simulations were performed, additional cases should be tested, such as repeated jump operations, consecutive store instructions, SKZ behavior when the accumulator is zero and non-zero, and boundary memory access at the lowest and highest addresses. These cases would help evaluate the robustness of the processor under less straightforward execution scenarios.

Finally, synthesis and implementation analysis should be performed to evaluate the hardware cost of the design. Resource utilization, critical path delay, maximum operating frequency, and power consumption should be analyzed after synthesis. This would provide a more complete understanding of the processor not only as a functional Verilog model, but also as a hardware design that could be implemented on an FPGA or similar digital platform.

Overall, this assignment helped us understand the fundamental principles of processor design, especially the interaction between datapath modules and control logic. It also provided practical experience in using Verilog HDL, designing a finite state machine controller, integrating multiple hardware modules, and verifying a digital system through simulation.

7 Member Contribution

Table 7.1: Member Contribution

No.	Member	Student ID	Contribution
1	Trần Vinh Quang	2550342	Implemented the Controller module, including the finite state machine and control signal generation. Contributed to the project report.
2	Lê Hiền Vinh	2453422	Implemented the Program Counter and top-level CPU modules. Developed the testbenches and expected output files, and performed simulation verification for the system.
3	Nguyễn Đức Thiên Tân	2550345	Implemented the Memory and Instruction Register modules, including memory access logic and instruction decoding. Contributed to the project report.
4	Đặng Ngọc Lan Anh	2550258	Implemented the ALU and Accumulator modules, including arithmetic, logic, zero flag, and register operations. Contributed to the project report.